BAI APPLICATION

EC2 Deployment & Installation Guide

Amazon Linux 2023 • React + Vite frontend • FastAPI backend • Supabase PostgreSQL

Deployment architecture

Browser	Nginx / port 80	Frontend files in /usr/share/nginx/html
Browser	FastAPI / port 8001	Direct API calls using VITE_API_BASE_URL
FastAPI	Supabase PostgreSQL	Database connection through Supabase pooler

This guide preserves the existing deployment approach: the browser loads the frontend from port 80 and calls the FastAPI backend directly on port 8001. It does not require changing the frontend to an Nginx /api reverse-proxy design.

Before starting, confirm the current EC2 Public IPv4 address in the AWS EC2 console. Public IPv4 addresses can change when an instance is stopped and started unless an Elastic IP is attached.

1. Prerequisites

- An Amazon Linux 2023 EC2 instance with SSH access.
- A security group allowing SSH (22) from your trusted IP, HTTP (80) from the intended audience, and TCP 8001 from the frontend users because the browser calls the API directly.
- GitHub access to https://github.com/Ashokk19/BAI.git.
- Supabase database connection details already available in the backend environment file.
- The current EC2 Public IPv4 address, referred to below as **EC2_PUBLIC_IP**.

Security: Do not expose database port 5432 to the internet. Do not paste database passwords, API keys, or the backend .env file into chat or commit them to Git.

2. Connect to EC2 and install system packages

```
sudo dnf update -y
sudo dnf install -y git nginx python3.11 python3.11-pip
sudo systemctl enable --now nginx
```

Install Node.js 22 (using NVM):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install 22
nvm use 22
node --version
npm --version
```

Expected major versions: Node.js 22 and npm 10 or later. If NVM is not available immediately, reconnect or run **source ~/.bashrc**.

3. Clone the application

```
cd /home/ec2-user
git clone https://github.com/Ashokk19/BAI.git
cd /home/ec2-user/BAI
```

If the repository already exists, update it instead:

```
cd /home/ec2-user/BAI
git pull
```

4. Configure and install the backend

Create the Python virtual environment

```
cd /home/ec2-user/BAI/backend
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If the application imports ReportLab and it is not already in requirements.txt, install it and add it to the dependency file so future deployments install it automatically:

```
pip install reportlab
pip freeze | grep -i reportlab
```

Configure backend environment variables

Create or edit the backend environment file. Preserve the existing Supabase credentials and all other application settings. Do not replace the file with only the CORS setting.

```
nano /home/ec2-user/BAI/backend/.env
```

Add or update the CORS variable with the browser's frontend origin. For the current example IP:

```
BAI_CORS_ORIGINS=http://16.113.84.245
```

The frontend origin is the address in the browser's address bar, including scheme and port if non-default. With Nginx serving HTTP on port 80, it is **http://16.113.84.245** — not the API address ending in :8001. Keep any existing application environment variables in this file.

5. Configure CORS in FastAPI

The browser considers the frontend and API different origins because their ports differ (80 and 8001). FastAPI must allow the frontend origin exactly. Use one shared list for both CORSMiddleware and any custom error-origin checks.

In **backend/app/main.py**, add the os import if it is not already present:

```
import os
```

Before the CORS helper and middleware configuration, define the allowed origins:

```
ALLOWED_CORS_ORIGINS = list(settings.ALLOWED_ORIGINS)

environment_origins = os.getenv("BAI_CORS_ORIGINS", "")
ALLOWED_CORS_ORIGINS.extend(
    origin.strip().rstrip("/")
    for origin in environment_origins.split(",")
    if origin.strip()
)
ALLOWED_CORS_ORIGINS = list(dict.fromkeys(ALLOWED_CORS_ORIGINS))
```

Update the custom origin helper to use the same list:

```
def _is_allowed_error_origin(origin: str | None) -> bool:
    if not origin:
        return False
    return origin.rstrip("/") in ALLOWED_CORS_ORIGINS
```

Configure CORSMiddleware with that same list. Retain the existing methods and headers used by your application:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"],
    allow_headers=["*"],
    expose_headers=["Content-Disposition", "Content-Type"],
    max_age=3600,
)
```

Important: Do not use allow_origins=["*"] together with allow_credentials=True. CORS origins must match exactly. If settings.ALLOWED_ORIGINS is not a Python list in your project configuration, adapt the initialization to its actual type before restarting.

6. Create the backend systemd service

Create the service file:

```
sudo nano /etc/systemd/system/bai-backend.service
```

Use the following service definition. Confirm the paths match the clone location above:

```
[Unit]
Description=BAI FastAPI Backend
After=network.target

[Service]
User=ec2-user
Group=ec2-user
WorkingDirectory=/home/ec2-user/BAI/backend
EnvironmentFile=/home/ec2-user/BAI/backend/.env
ExecStart=/home/ec2-user/BAI/backend/.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8001
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Load and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now bai-backend
sudo systemctl status bai-backend
```

Check backend logs:

```
sudo journalctl -u bai-backend -n 100 --no-pager
```

The backend must bind to **0.0.0.0:8001** for direct browser access. Binding only to 127.0.0.1 would make it reachable only from the EC2 instance itself.

7. Configure Nginx to serve the frontend

Nginx serves the built frontend on port 80. On Amazon Linux, the default server configuration commonly already contains the document root /usr/share/nginx/html and SPA fallback. Inspect it first:

```
sudo nginx -T | less
```

Ensure the active server block serves /usr/share/nginx/html and routes unknown frontend paths to index.html. A typical location block is:

```
location / {
    try_files $uri $uri/ /index.html;
}
```

Avoid adding a second conflicting root or index directive in /etc/nginx/default.d/*.conf if those directives already exist in the parent server block. Validate and reload:

```
sudo nginx -t
sudo systemctl enable nginx
sudo systemctl restart nginx
curl -I http://127.0.0.1/
```

8. Build and deploy the frontend

Set the API base URL to the EC2 public IP and port 8001 before running the production build:

```
cd /home/ec2-user/BAI/frontend
export VITE_API_BASE_URL="http://16.113.84.245:8001"
npm install
npm run build
```

Deploy the build output to Nginx:

```
sudo rm -rf /usr/share/nginx/html/*
sudo cp -r dist/* /usr/share/nginx/html/
sudo chown -R nginx:nginx /usr/share/nginx/html
sudo systemctl restart nginx
```

Vite embeds VITE_API_BASE_URL at build time. If this value changes, rebuild the frontend and redeploy the dist directory. Changing only the backend environment file does not update the frontend API URL.

9. Verify the deployment

On the EC2 instance

```
# Frontend served by Nginx
curl -I http://127.0.0.1/

# FastAPI OpenAPI endpoint
curl -I http://127.0.0.1:8001/openapi.json

# Check service and listening ports
sudo systemctl status nginx
sudo systemctl status bai-backend
sudo ss -lntp | grep -E ':80|:8001'
```

Test the CORS preflight

Run this on EC2, replacing the origin with the exact frontend address:

```
curl -i -X OPTIONS \
  http://127.0.0.1:8001/api/accounts/public \
  -H "Origin: http://16.113.84.245" \
  -H "Access-Control-Request-Method: GET"
```

A successful preflight should return an allowed-origin header matching **http://16.113.84.245** and a successful preflight status (normally 200 or 204). A 400 response or missing Access-Control-Allow-Origin means the origin is still not in the effective allowlist.

From your Windows computer

```
Test-NetConnection 16.113.84.245 -Port 80
Test-NetConnection 16.113.84.245 -Port 8001
```

Open **http://16.113.84.245** in the browser. Use HTTP until you have configured TLS/HTTPS and a certificate. Do not try https:// on port 443 before HTTPS is configured.

10. If the EC2 public IP changes

Update both the backend CORS allowlist and the frontend's build-time API URL. Example: if the new public IP is NEW_PUBLIC_IP:

1. Edit **/home/ec2-user/BAI/backend/.env** and set **BAI_CORS_ORIGINS=http://NEW_PUBLIC_IP**. Keep the rest of the .env file unchanged.

2. Restart the backend so systemd reloads the environment file: **sudo systemctl restart bai-backend**.

3. Rebuild the frontend with **export VITE_API_BASE_URL="http://NEW_PUBLIC_IP:8001"**.

4. Copy the new dist output to /usr/share/nginx/html and restart Nginx.

5. Update any browser bookmarks, external integrations, or security-group source rules that refer to the old IP.

6. Repeat the CORS preflight and browser tests in Section 9.

```
# Backend environment
sudo nano /home/ec2-user/BAI/backend/.env
# Set:
# BAI_CORS_ORIGINS=http://NEW_PUBLIC_IP

sudo systemctl restart bai-backend

# Frontend rebuild
cd /home/ec2-user/BAI/frontend
export VITE_API_BASE_URL="http://NEW_PUBLIC_IP:8001"
npm run build
sudo rm -rf /usr/share/nginx/html/*
sudo cp -r dist/* /usr/share/nginx/html/
sudo chown -R nginx:nginx /usr/share/nginx/html
sudo systemctl restart nginx
```

Recommended: Allocate and associate an AWS Elastic IP to the EC2 instance if you need a stable public IPv4 address. This avoids repeating the IP update procedure after ordinary stop/start cycles. Elastic IPs may have AWS charges depending on usage and current pricing.

11. Security group guidance

Port	Protocol	Suggested source	Purpose
22	TCP	Your trusted public IP /32	SSH administration
80	TCP	0.0.0.0/0 if public website	Frontend HTTP
8001	TCP	Required client IP ranges	Direct FastAPI access from browser
443	TCP	0.0.0.0/0 when TLS is configured	HTTPS

Because this deployment intentionally uses direct browser-to-API calls on port 8001, that port must be reachable from the users' browsers. Restrict its source range where practical. Do not open port 5432 for Supabase/PostgreSQL.

12. Troubleshooting

Frontend loads but API calls fail

Check the browser Network tab. Confirm the request URL uses the current public IP and :8001. Test port 8001 from your computer and inspect the backend logs.

CORS error / preflight returns 400

Confirm BAI_CORS_ORIGINS contains the frontend origin exactly, including http/https and any non-default port. Restart bai-backend and rerun the OPTIONS test.

API works with curl on EC2 but not from browser

Check the EC2 security group and host firewall for TCP 8001. Confirm Uvicorn uses --host 0.0.0.0.

Database connection refused to localhost:5432

Check the backend .env database URL. It should point to the configured Supabase pooler/host, not localhost, unless PostgreSQL is intentionally installed locally.

Backend service fails after reboot

Run systemctl status bai-backend and journalctl -u bai-backend -n 100. Check WorkingDirectory, venv path, requirements, and EnvironmentFile.

Frontend changes do not appear

Re-run npm run build and copy the new dist contents to /usr/share/nginx/html. Vite environment variables are embedded during build.

HTTPS connection refused

HTTPS is not configured merely because HTTP works. Configure a domain, TLS certificate, and Nginx 443 listener before using https://.

13. Deployment checklist

- EC2 instance has Git, Nginx, Python 3.11, and Node.js 22 installed.
- Repository cloned under /home/ec2-user/BAI.
- Backend virtual environment and requirements installed.
- Backend .env contains the correct Supabase connection settings and BAI_CORS_ORIGINS.
- FastAPI CORS middleware and custom origin helper use ALLOWED_CORS_ORIGINS.
- bai-backend systemd service is enabled and running on 0.0.0.0:8001.
- Nginx serves the frontend on port 80 with SPA fallback.
- Frontend built with VITE_API_BASE_URL pointing to http://EC2_PUBLIC_IP:8001.
- Security group allows only the necessary inbound traffic.
- CORS preflight, API connectivity, and browser application flows tested.

Deployment model to retain: Nginx serves the React/Vite frontend on port 80; the browser calls FastAPI directly at http://EC2_PUBLIC_IP:8001; FastAPI connects to Supabase PostgreSQL.